

ASTra — Complete Viva Preparation Guide

AST-Native Code Intelligence / Final Year Project

1. What Is ASTra?

ASTra (AST-Native Code Intelligence) is a static code analysis platform that reads source code without running it and tells you:

- How fast the code is (Time Complexity: $O(n)$, $O(n^2)$...)
- How much memory it uses (Space Complexity)
- Where bugs and inefficiencies hide
- How complex it is to test (Cyclomatic Complexity)
- Whether there is dead code, anti-patterns, or type issues

One-Line Answer: "ASTra is a full-stack static code intelligence platform that analyzes source code using 10 custom-built AST engines to infer time/space complexity, detect bugs and anti-patterns, visualize control flow, and provide AI-assisted explanations — all without executing the code."

2. Why Does It Exist?

Problem	How ASTra Solves It
Developers don't know their code's complexity	ASTra infers $O(n)$, $O(n^2)$ etc. automatically
ESLint/PyLint only catch syntax/style issues	ASTra does semantic analysis — data flow, recursion, Halstead metrics
Cloud AI tools need internet + paid APIs	ASTra's 10 core engines run 100% locally
No tool explains WHY complexity is what it is	ExplanationBuilder gives plain-English reasoning with your actual variable names
No interactive way to practice efficient algorithms	ASTra has Challenge Mode with 10 graded challenges

3. How It Is Different From Others

Feature	ESLint/PyLint	ChatGPT/Copilot	ASTra
Time complexity detection	No	Estimates only	Yes — precise, deterministic
Works fully offline	Yes	No	Yes
Halstead Metrics	No	No	Yes
Cyclomatic Complexity	Limited	No	Yes — per function
CFG Visualization	No	No	Yes — interactive SVG
Dead code detection	Partial	No	Yes — 5 patterns
Type inference (no execution)	No	No	Yes

Challenge Mode with grading	No	No	Yes — 10 challenges
ML quality prediction	No	No	Yes — Random Forest
Carbon footprint Eco Score	No	No	Yes
Built from scratch, no linter libs	N/A	N/A	Yes — all 10 engines

4. Tech Stack

Layer	Technology	Why
Frontend	React 18 + Vite	Fast SPA, hot reload, component-based
Styling	Custom CSS (glassmorphism)	Full control, no framework dependency
Backend	Flask (Python)	Lightweight, perfect for REST API + ML
Analysis — Python	Python's built-in ast module	Native AST access, no external parser
Analysis — other langs	Tree-sitter	Universal parser for JS, Java, C, C++
Database	SQLite + SQLAlchemy	Zero-config, file-based
Authentication	JWT + bcrypt	Stateless auth, secure password hashing
AI Chat	HuggingFace API (SSE)	Free-tier LLM, streamed responses
ML Model	Random Forest + scikit-learn	Interpretable, fast, trained on MBPP
Security	flask-talisman, flask-limiter	CSP headers, rate limiting
Containerization	Docker + docker-compose	One-command deployment
CI	GitHub Actions	Automated tests on every push

5. System Architecture

Request Flow: User pastes code → React sends POST /analyze → Flask backend detects language → All 10 engines run on AST (Python) → Random Forest predicts quality → Eco score + patterns computed → Saved to SQLite (SHA-256 dedup) → JSON response → React renders dashboard.

Run commands: Backend: `cd backend && python app.py` (<http://localhost:5000>) | Frontend: `cd frontend && npm run dev` (<http://localhost:5173>) | Docker: `docker-compose up --build`

6. The 10 Analysis Engines

Engine 1 — InferenceEngine

Infers time AND space complexity by walking the AST. Finds For/While nodes, classifies each, multiplies when nested. Example: two nested for-loops over $n \rightarrow O(n^2)$. while $i \neq 2 \rightarrow O(\log n)$.

Engine 2 — RecursionClassifier

Classifies 6 recursion patterns: linear $f(n-1)$, binary $f(n/2)$, divide-and-conquer ($2 \times f(n/2)$), tail-recursive, memoized (`@lru_cache`), mutual recursion. Each gets a complexity hint.

Engine 3 — ExplanationBuilder

Generates plain-English complexity explanations using your actual variable names. Fully deterministic — zero LLM involved. Example: 'The outer loop on line 3 iterates over nums ($O(n)$). Inner loop also $O(n)$. Combined: $O(n^2)$.'

Engine 4 — DataFlowTracer

Detects 5 data-flow inefficiency patterns: (1) x in list instead of set — $O(n)$ vs $O(1)$, (2) string concat in loop — quadratic, (3) repeated `len()` calls in loop, (4) sort-then-index instead of `min()/max()`, (5) nested list comprehensions — $O(n^2)$ memory.

Engine 5 — AntiPatternDetector

Catches 5 classic Python bugs: (1) mutable default arguments `def f(x=[])`, (2) bare `except:` silently swallows all errors, (3) global variable modification inside functions, (4) `return` inside a loop, (5) redundant constant assignments.

Engine 6 — CyclomaticAnalyzer

Implements McCabe's 1976 formula. Counts decision points (`if/elif/for/while/except/and/or`) + 1. Risk labels: Simple(1-4), Moderate(5-7), Complex(8-10), High(11-15), Untestable(16+).

Engine 7 — ConfidenceEstimator

Adds honest uncertainty bounds to every estimate. If loop bound is `range(f(x))` — can't determine statically → confidence drops below 0.85. No other tool admits uncertainty like this.

Engine 8 — DeadCodeDetector

Detects 5 dead code patterns: unused variables, unused imports, unreachable code after `return/raise`, functions never called, `if False:` branches.

Engine 9 — TypeInferencer

Infers variable types statically without running code. `x=[]` → list, `x={}` → dict, `x='hi'` → str. Feeds into DataFlowTracer for accurate pattern detection.

Engine 10 — HalsteadAnalyzer

Implements Halstead's 1977 metrics: Vocabulary = unique operators+operands, Volume = $N \times \log_2(\text{vocabulary})$, Difficulty = how hard to understand, Effort = Difficulty×Volume, Bugs Estimated = Volume/3000.

7. Languages Supported

Language	Parser	Analysis Depth
Python	Built-in ast module	Full — all 10 engines
JavaScript	Tree-sitter	Complexity + recursion + HOF methods
Java	Tree-sitter	Complexity + per-method breakdown
C	Tree-sitter	Complexity + pointer/malloc tracking
C++	Tree-sitter	Complexity + full STL awareness

Python gets deeper analysis because ast gives semantic context (variable names, types, scope). Tree-sitter gives structural parsing only.

8. ML Model

Property	Value
Algorithm	Random Forest Classifier + StandardScaler
Training data	Google MBPP dataset (~500 real Python programs)
Features	loop_depth, is_recursive, uses_extra_memory, time_penalty, space_penalty
Output	Binary — efficient (0) or inefficient (1)
Known limitation	Labels are test pass/fail proxy — correctness, not true efficiency

9. Security Model

Threat	Mechanism
XSS attacks	DOMPurify on all AI-generated markdown
Brute force	Rate limiting: 30/min on /analyze, 200/day global
Password theft	bcrypt hashing — never stored plain
Session hijacking	JWT tokens with 7-day expiry
Clickjacking	flask-talisman: CSP, HSTS, X-Frame-Options
Stack trace leaks	Global exception handler hides internals
Secrets in code	All via .env environment variables
Oversized input	Max 10,000 characters per submission

10. Challenge Mode

- 10 algorithmic challenges: Find Duplicates, Binary Search, Two Sum, Word Frequency, Flatten, Anagram, First Unique Char, Max Subarray, Group Anagrams, Longest Consecutive
- User writes Python code in the editor
- ASTra runs the code against test cases in a daemon thread (2-second timeout)
- ASTra also analyzes the complexity of the submitted code
- Grade: PASS (correct + efficient) | PARTIAL (correct but too slow) | FAIL (wrong answer)

11. Key API Endpoints

Endpoint	Method	Description
/analyze	POST	Full analysis — all 10 engines
/cfg	POST	Control Flow Graph (Python only)
/report	POST	Download plain-text analysis report
/chat	POST	SSE-streamed AI conversation
/history	GET	Last 50 analyses (JWT required)
/auth/register	POST	Create account
/auth/login	POST	Login, receive JWT
/challenges	GET	List all 10 challenges
/challenges/<id>/submit	POST	Grade challenge submission
/health	GET	Server health + ML model status

35 Viva Questions & Answers

Section A — Basic Understanding

Q1. What does ASTra stand for and what is it?

ASTra = AST-Native Code Intelligence. It is a static code analysis platform using 10 custom-built AST engines to infer complexity, detect bugs, and provide AI explanations without executing the code.

Q2. What is an Abstract Syntax Tree (AST)?

An AST is a tree representation of source code structure. Each node represents a code construct — a loop, function call, assignment. Python's ast module parses Python into this tree. ASTra walks this tree to extract complexity and quality information.

Q3. What is static analysis vs dynamic analysis?

Static analysis examines code without running it — like reading a recipe without cooking. Dynamic analysis runs the code and observes runtime behavior. Static analysis is safer, faster, and works before deployment.

Q4. Why build ASTra instead of using existing tools?

ESLint/Pylint only catch syntax/style issues. Copilot needs internet and paid APIs and gives non-deterministic answers. ASTra provides deterministic, precise complexity inference from first principles — every engine is built from scratch. It also adds features no single tool has: CFG visualization, Halstead metrics, confidence intervals, and a challenge mode.

Section B — Architecture & Technical

Q5. Explain the full flow of an analysis request.

1. User submits code → POST /analyze. 2. Backend detects language. 3. For Python: AST parsed; all 10 engines run. 4. Features extracted → Random Forest predicts quality. 5. Eco score, patterns, suggestions computed. 6. Result saved to SQLite with SHA-256 dedup. 7. JSON response → React renders the dashboard.

Q6. Why Flask over Django or FastAPI?

Flask is a micro-framework giving fine-grained control. We didn't need Django's admin or ORM (we use SQLAlchemy directly). FastAPI would also have been good (better async), but Flask integrates cleanly with Python's ast module and our scikit-learn ML pipeline.

Q7. Why SQLite and not PostgreSQL?

SQLite is file-based and zero-config — ideal for a final year project. For production scale we would migrate to PostgreSQL, but SQLite handles our load perfectly.

Q8. How does language auto-detection work?

language_detector.py uses weighted heuristic scoring. It scans for language-specific keywords (def/: for Python, function/=> for JS, public class for Java, #include for C/C++) and assigns weighted scores. The highest scorer wins.

Q9. What is Tree-sitter?

Tree-sitter is a universal parser generator providing concrete syntax trees for many languages. ASTra uses it for JS, Java, C, and C++ because Python's ast only handles Python. Tree-sitter gives structural parse trees; ASTra's analyzers walk them to extract patterns.

Section C — Analysis Engines

Q10. How does ASTra determine $O(n^2)$?

InferenceEngine walks the AST and finds all For/While nodes. Each is classified (constant, linear, logarithmic). When nested, complexities are multiplied via _combine_nested(). Two nested linear loops $\rightarrow O(n) \times O(n) = O(n^2)$.

Q11. How does ASTra detect $O(\log n)$?

Two ways: (1) A while loop where the counter is multiplicatively modified ($i /= 2$) $\rightarrow O(\log n)$. (2) RecursionClassifier detects $f(n/2)$ (binary recursion) $\rightarrow O(\log n)$ hint.

Q12. What is Cyclomatic Complexity?

Introduced by Thomas McCabe in 1976. Measures linearly independent paths through code. Formula: count decision points (if/elif/for/while/except/and/or) + 1. Scale: Simple(1-4), Moderate(5-7), Complex(8-10), High(11-15), Untestable(16+).

Q13. What are Halstead Metrics?

Halstead's 1977 software science metrics: Volume = size in information bits, Difficulty = how hard to write/understand, Bugs Estimated = Volume/3000 (predicted defect count). Computed by counting unique and total operators and operands.

Q14. What is the ConfidenceEstimator doing?

Adds uncertainty bounds to estimates. If a loop bound is $\text{range}(f(x))$ — we can't statically determine if $f(x)$ is constant or variable — confidence drops below 0.85 and an alternative suggestion is shown. No other tool admits uncertainty like this.

Q15. Explain DataFlowTracer with an example.

Traces data flow to find inefficiency patterns. Example: 'x in my_list' inside a loop — TypeInferencer confirms my_list is a list $\rightarrow$ DataFlowTracer flags it because list membership is $O(n)$ vs $O(1)$ for a set. Converting to a set would fix it.

Q16. What 5 anti-patterns does ASTra detect?

1. Mutable default arguments $\text{def } f(x=[])$ — default list shared across all calls. 2. Bare except: — catches everything including KeyboardInterrupt. 3. Global variable

modification inside functions. 4. Return inside a loop — premature exit bug. 5. Redundant constant assignments.

Section D — ML & AI

Q17. What ML model does ASTra use and why Random Forest?

Random Forest Classifier with StandardScaler. Chosen because: interpretable (feature importance), handles small datasets well (~500 MBPP samples), robust to noisy features, fast prediction, and needs no hyperparameter tuning for a baseline.

Q18. What is the Google MBPP dataset?

MBPP = Mostly Basic Python Problems. A Google dataset of ~500 crowd-sourced Python problems with test cases. ASTra uses it to train the Random Forest by extracting features (loop depth, recursion, extra memory) and using test pass/fail as an efficiency proxy label.

Q19. What is the ML model's limitation?

The label proxy is imperfect. A correct-but-slow $O(n^2)$ solution passes tests → labeled 'efficient'. A fast $O(n)$ solution with a bug fails tests → labeled 'inefficient'. The model actually measures correctness, not true algorithmic efficiency.

Q20. How does the AI chat work?

User message + current analysis result (as system context) → POST /chat → backend calls HuggingFace Inference API → response streamed via Server-Sent Events (SSE) → frontend reads chunks and renders in real-time. DOMPurify sanitizes all output to prevent XSS.

Section E — Security

Q21. How are passwords stored?

Using bcrypt hashing. Bcrypt has a built-in salt and is intentionally slow (cost factor), making brute-force attacks computationally expensive. Passwords are never stored in plaintext.

Q22. What is JWT and how does ASTra use it?

JWT = JSON Web Token. After login, the backend issues a signed token with the user's ID and 7-day expiry. The frontend sends it as 'Authorization: Bearer <token>' with protected requests. The backend verifies the signature without hitting the database.

Q23. What is CSP?

Content Security Policy — an HTTP header telling the browser which content sources are allowed. ASTra via flask-talisman only allows content from self and HuggingFace. Prevents XSS attacks where attackers inject malicious scripts.

Q24. Why rate limiting?

Restricts requests to 30/minute per IP on /analyze. Prevents abuse (server flooding), protects the HuggingFace API quota, and ensures fair usage for all users.

Q25. How does SHA-256 deduplication work?

Before saving, the backend hashes the submitted code with SHA-256. If the same code was analyzed before, the hash already exists — no duplicate record is created. Keeps history clean and database small.

Section F — Frontend & UX

Q26. What is Vite?

A modern JavaScript build tool using ES modules for near-instant hot module replacement during development. Much faster than webpack-based Create React App. Bundles the frontend into optimized static files for production.

Q27. What is the CFG Visualizer?

CFG = Control Flow Graph. Nodes are code blocks; edges are execution paths (true/false branches, loops, exceptions). CFGBuilder generates this for Python. CFGVisualizer.jsx renders it as an interactive SVG with pan/zoom and color-coded edge types.

Q28. What is SSE (Server-Sent Events)?

SSE is a web technology where the server pushes data to the client over a persistent HTTP connection — one-way (server to client). ASTra uses SSE for AI chat to stream text tokens to the browser, creating a typewriter effect.

Q29. What is the Diff Analyzer?

DiffAnalyzer.jsx allows side-by-side comparison of two code snippets. Both are analyzed independently and their complexity metrics compared — useful for demonstrating improvement from $O(n^2)$ to $O(n)$.

Section G — Project-Level

Q30. What were the biggest challenges?

Writing InferenceEngine's nested loop multiplication logic (all edge cases), making Tree-sitter work across 4 languages with consistent output, CFG Visualizer layout algorithm (preventing node overlap), and security hardening (setting CSP without breaking HuggingFace SSE streaming).

Q31. How did you test the project?

pytest test suite in backend/tests/, test_languages.py smoke test for all 5 language analyzers, GitHub Actions CI runs tests on every push, and manual testing via the UI.

Q32. Can ASTra detect all complexity classes?

ASTra detects: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$. Limitation: binary search via lo/hi convergence is not detected for Python — that requires abstract interpretation, a research-level problem.

Q33. What would you improve given more time?

Add abstract interpretation for precise binary search detection, extend Halstead/dead code/type inference to JS/Java/C/C++, Docker-isolated sandboxing for Challenge Mode, support more languages (Rust, Go), PostgreSQL for production scalability.

Q34. What is the Eco Score?

eco_score.py estimates carbon footprint based on time/space complexity and language. Higher complexity = more CPU = more energy = higher CO₂. Outputs an Eco Score (0-100, higher is greener) and CO₂ estimate. Promotes sustainable coding practices.

Q35. What does the name ASTra mean?

AST = Abstract Syntax Tree — the core data structure all 10 engines operate on. 'ra' completes the word. Together it means 'AST-native' — the analysis lives in the AST itself, not bolted onto an existing linter.

Quick Reference Card — Memorize These

Item	Value
Number of engines	10
Languages supported	5 (Python, JS, Java, C, C++)
Python parser	Built-in ast module
Other language parser	Tree-sitter
Backend	Flask
Frontend	React 18 + Vite
Database	SQLite + SQLAlchemy
Auth	JWT + bcrypt
ML algorithm	Random Forest Classifier
Training data	Google MBPP dataset
Rate limit on /analyze	30 requests/minute
Max code input	10,000 characters
JWT expiry	7 days
Challenge problems	10
Challenge timeout	2 seconds per test
Cyclomatic formula	Decision points + 1
Halstead bug formula	Volume / 3000
Confidence threshold	0.85

Presentation Tips

- Open with the problem: 'Every developer writes code but rarely knows its true efficiency. ASTra changes that.'
- Live demo flow: Paste a nested loop → show $O(n^2)$ → paste optimized version → show $O(n)$. The visual improvement is dramatic.
- If asked something you don't know: Say 'That's actually one of our known limitations and a clear future improvement.'
- Emphasize the from-scratch claim: 'We wrote 10 engines from scratch. We didn't use Pylint, we didn't use Radon — every rule is our own.'
- Most impressive feature to demo: CFG Visualizer — visually striking and technically complex. Show it last for maximum impact.

Good luck tomorrow! You have built something genuinely impressive — own it confidently. 🚀